lightning talks (10- to 15-minute presentations) by domain experts on topics such as the business case, competing products, and user profiles. This information is captured on whiteboards (e.g., as customer journey maps, personas, or user task workflow) and remains posted for easy reference throughout the sprint.

- **Sketch.** Individual team members (including all stakeholders) are given the time and space needed to brainstorm solutions to the problems discovered in the understand phase. It is best to do this on paper as quick visual images. Paper drawings and notes are easy to generate, easy to modify, and quite inexpensive. This phase generates lots of ideas because participant ideas are not restricted when creating their solutions.
- **Decide.** Each stakeholder presents his solution sketch, and the team votes to determine the solutions that should be tackled in the prototyping activities that will follow. If there is not a clear consensus following the voting, the development team may decide to consider assumptions that involve constraints posed by the budget, user profiles, available resources (both human and technological), and product business goals.
- **Prototype.** The prototype created during this phase may be a minimally viable product based on the solution selected from the sketch phase, or it may be based on the portions of the customer journey map or storyboard you want to evaluate with potential users in the validate phase. Think of your prototype as an experiment developed to test a hypothesis. That means the team should be developing test cases based on the user stories as the prototype is being built. There is no need to create a fully functional backend for this user interface prototype. It would be best to build a digital prototype using a simple tool (e.g., Keynote[5]). In some cases it may be desirable to create a paper prototype using one of the developers to provide the screen sequences to the users if a prototyping tool is not available.
- **Validate.** Watching users try out your prototype is the best way to discover major issues with its UX design, which in turn lets you start iterating immediately. In a UX design sprint everyone on the development team observes the validation sessions, not just the UX expert or test case designer. This is critical to capturing potential learning opportunities by exposing product decision makers to user feedback in real time. We will talk more about prototype reviews and user testing in Section 12.7.

12.6 User Interface Design

The definition of interface objects and the actions that are applied to them is an important step in interface design. To accomplish this, user scenarios are parsed in much the same way as described in Chapter 9. That is, a use case is written. Nouns (objects) and verbs (actions) are isolated to create a list of screen objects and systems actions.

5 See https://keynotopia.com/.

Once the objects and actions have been defined and elaborated iteratively, they are categorized by type. Target, source, and application objects are identified. A *source object* (e.g., a sensor icon) is dragged and dropped onto a *target object* (e.g., a screen location). The implication of this action is to place a sensor on the room floor plan. An *application object* represents application-specific data that are not directly manipulated as part of screen interaction. For example, the code used to simulate the coverage of the room sensors allow for testing of the sensor placement. The code would be associated with each sensor through some type of logical link when the object is placed on the screen, but the code itself is not dragged and dropped via user interaction.

When you are satisfied that all important objects and actions have been defined (for one design iteration), screen layout is performed. Like other interface design activities, screen layout is an interactive process in which graphical design and placement of icons, definition of descriptive screen text, specification and titling for windows, and definition of major and minor menu items are conducted. If a real-world metaphor is appropriate for the application, it is specified at this time, and the layout is organized in a manner that complements the metaphor.

12.6.1 Applying Interface Design Steps

To provide a brief illustration of how design might proceed to create a prototype of a user interface, let's consider a user scenario for the *SafeHome* system (discussed in earlier chapters). A preliminary user story (written by the homeowner) for the interface follows:

> **Preliminary user story:** I want to gain access to my *SafeHome* system from any remote location via the Internet. Using browser software operating on my mobile device (while I'm at work or traveling), I can determine the status of the alarm system, arm or disarm the system, reconfigure security zones, and view different rooms within the house via preinstalled video cameras.
>
> To access *SafeHome* from a remote location, I provide an identifier and a password. These define levels of access (e.g., all users may not be able to reconfigure the system) and provide security. Once validated, I can check the status of the system and change the status by arming or disarming *SafeHome.* I can reconfigure the system by displaying a floor plan of the house, viewing each of the security sensors, displaying each currently configured zone, and modifying zones as required. I can view the interior of the house via strategically placed video cameras. I can pan and zoom each camera to provide different views of the interior.

Based on this user story, the following homeowner tasks, objects, and data items are identified:

- *Accesses* the *SafeHome* system
- *Enters* an **ID** and **password** to allow remote access
- *Checks* **system status**
- *Arms* or *disarms SafeHome* system
- *Displays* **floor plan** and **sensor locations**
- *Displays* **zones** on floor plan
- *Changes* **zones** on floor plan
- *Displays* **video camera locations** on floor plan

- *Selects* **video camera** for viewing
- *Views* **video images** (four frames per second)
- *Pans* or *zooms* the **video camera**

Objects (boldface) and actions (italics) are extracted from this list of homeowner tasks. The majority of objects noted are application objects. However, **video camera location** (a source object) is dragged and dropped onto **video camera** (a target object) to create a **video image** (a window with video display).

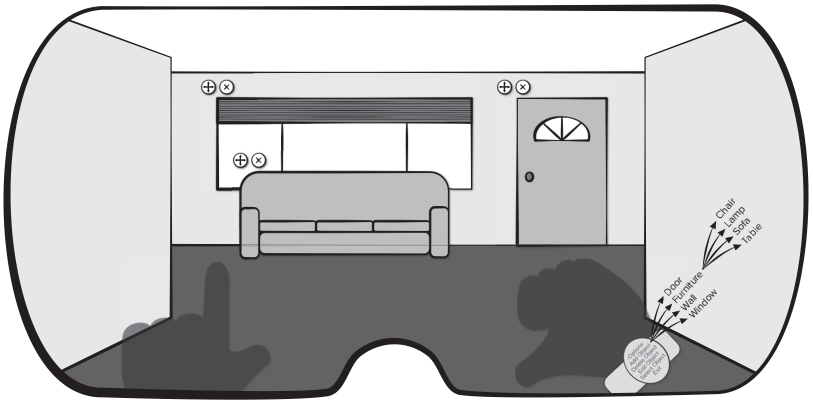
A preliminary sketch of the screen layout for video monitoring is created (Figure 12.6).[6] To invoke the video image, a video camera location icon, *C,* located in the floor plan displayed in the monitoring window is selected. In this case a camera location in the living room (LR) is then dragged and dropped onto the video camera icon in the upper-left-hand portion of the screen. The video image window appears, displaying streaming video from the camera located in the LR. The zoom and pan control slides are used to control the magnification and direction of the video image. To select a view from another camera, the user simply drags and drops a different camera location icon into the camera icon in the upper-left-hand corner of the screen.

The layout sketch shown would have to be supplemented with an expansion of each menu item within the menu bar, indicating what actions are available for the video monitoring mode (state). A complete set of sketches for each homeowner task noted in the user scenario would be created during the interface design.

12.6.2 User Interface Design Patterns

Graphical user interfaces have become so common that a wide variety of user interface design patterns has emerged. A *design pattern* is an abstraction that prescribes a design solution to a specific, well-bounded design problem.

FIGURE 12.6
Preliminary screen layout



6 Note that this differs somewhat from the implementation of these features in earlier chapters. This might be considered a first-draft design based on the new room layout app.

As an example of a commonly encountered interface design problem, consider a situation in which a user must enter one or more calendar dates, sometimes months in advance. There are many possible solutions to this simple problem, and a number of different patterns that might be proposed. Laakso [Laa00] suggests a pattern called **CalendarStrip** that produces a continuous, scrollable calendar in which the current date is highlighted and future dates may be selected by picking them from the calendar. The calendar metaphor is well known to every user and provides an effective mechanism for placing a future date in context.

A vast array of interface design patterns has been proposed over the past few decades.[7] A more detailed discussion of user interface design patterns is presented in Chapter 14. In addition, Punchoojit [Pun17] provides a systematic review of many mobile device user interface design patterns (e.g., zooming, lateral access, revealing information in context, control, and conformation).

12.7 Design Evaluation

Once you create an operational user interface prototype, it must be evaluated to determine whether it meets the needs of the user. Evaluation can span a formality spectrum that ranges from an informal "test drive," in which a user provides impromptu feedback to a formally designed study that uses statistical methods for the evaluation of questionnaires completed by a population of end users.

The user interface evaluation cycle takes the form shown in Figure 12.7. After the design model has been completed, a first-level prototype is created. The prototype is evaluated by the user,[8] who provides you with direct comments about the efficacy of the interface. In addition, if formal evaluation techniques are used (e.g., questionnaires, rating sheets), you can extract information from these data (e.g., 80 percent of all users did not like the mechanism for saving data files). Design modifications are made based on user input, and the next level prototype is created. The evaluation cycle continues until no further modifications to the interface design are necessary. We discuss specialized prototype review and testing techniques for graphical user interfaces in Chapter 21.

12.7.1 Prototype Review

The prototyping approach is effective, but is it possible to evaluate the quality of a user interface before a prototype is built?[9] If you identify and correct potential problems early, the number of loops through the evaluation cycle will be reduced and development time will shorten. If a design model (user stories, storyboard, personas,

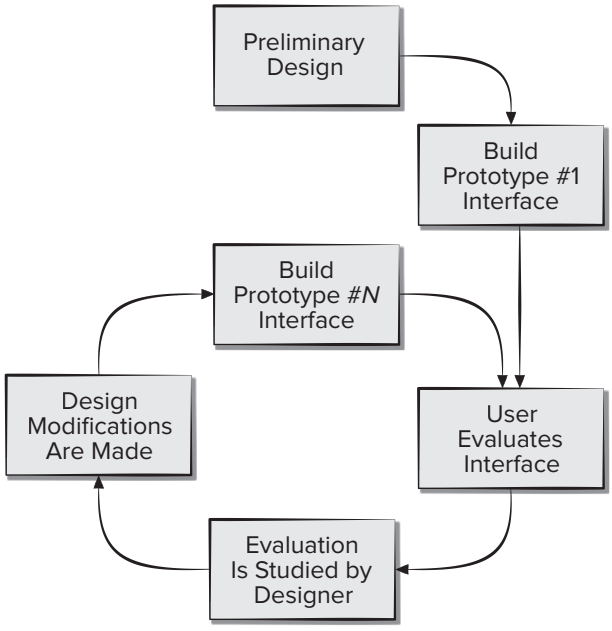
7 Useful suggestions for sites that address user interface patterns can be found at https://www.interaction-design.org/literature/article/10-great-sites-for-ui-design-patterns.

8 It is important to note that experts in ergonomics and interface design may also conduct reviews of the interface. These reviews are called *heuristic evaluations* or *cognitive walk-throughs.*

9 Some software engineers prefer to develop a low-fidelity mockup of the user interface (UI) called a paper prototype to allow stakeholders to test the UI concept before committing any programming resources. The process is described here: http://www.paperprototyping.com/what_examples.html.

FIGURE 12.7

The interface design evaluation cycle



etc.) of the interface has been created, a number of evaluation criteria [Mor81] can be applied during early design reviews:

1. The length and complexity of the requirements model or written specification of the system and its interface provide an indication of the amount of learning required by users of the system.
2. The number of user tasks specified and the average number of actions per task provide an indication of interaction time and the overall efficiency of the system.
3. The number of actions, tasks, and system states indicated by the design model imply the memory load on users of the system.
4. Interface style, help facilities, and error-handling protocol provide a general indication of the complexity of the interface and the degree to which it will be accepted by the user.

SAFEHOME



Interface Design Review

The scene: Doug Miller's office.

The players: Doug Miller, manager of the *SafeHome* software engineering group, and Vinod Raman, a member of the *SafeHome* product software engineering team.

The conversation:

Doug: Vinod, have you and the team had a chance to review the **SafeHomeAssured.com** e-commerce interface prototype?

Vinod: Yeah . . . we all went through it from a technical point of view, and I have a bunch of notes. I e-mailed 'em to Sharon [manager of the WebApp team for the outsourcing vendor for the *SafeHome* e-commerce website] yesterday.

Doug: You and Sharon can get together and discuss the small stuff . . . give me a summary of the important issues.

Vinod: Overall, they've done a good job. Nothing ground breaking, but it's a typical e-commerce interface, decent aesthetics, reasonable layout; they've hit all the important functions . . .

Doug (smiling ruefully): But?

Vinod: Well, there are a few things. . . .

Doug: Such as?

Vinod (showing Doug a sequence of storyboards for the interface prototype): Here's the major functions menu that's displayed on the home page:

Learn about *SafeHome*.

Describe your home.

Get *SafeHome* component recommendations.

Purchase a *SafeHome* system.

Get technical support.

The problem isn't with these functions. They're all okay, but the level of abstraction isn't right.

Doug: They're all major functions, aren't they?

Vinod: They are, but here's the thing . . . you can purchase a system by inputting a list of components . . . no real need to describe the house if you don't want to. I'd suggest only four menu options on the home page:

Learn about *SafeHome*.

Specify the *SafeHome* system you need.

Purchase a *SafeHome* system.

Get technical support.

When you select **Specify the *SafeHome* system you need**, you'll then have the following options:

Select *SafeHome* components.

Get *SafeHome* component recommendations.

If you're a knowledgeable user, you'll select components from a set of categorized pull-down menus for sensors, cameras, control panels, and more. If you need help, you'll ask for a recommendation and that will require that you describe your house. I think it's a bit more logical.

Doug: I agree. Have you talked with Sharon about this?

Vinod: No, I want to discuss this with marketing first; then I'll give her a call.

12.7.2 User Testing

Once the first interactive prototype is built, you can collect a variety of qualitative and quantitative data that will assist in evaluating the interface. To collect qualitative data, questionnaires that allow users to assess the interface prototype can be distributed. If quantitative data are desired, a form of time-study analysis can be conducted. Users are observed during interaction, and data—such as number of tasks correctly completed over a standard time period, frequency of actions, sequence of actions, time spent "looking" at the display, number and types of errors, error recovery time, time spent using help, and number of help references per standard time period—are collected and used as a guide for interface modification.